

Fixing Metastability Using a 2-Stage Synchronizer

Introduction:

Metastability is a reliability hazard that arises when an asynchronous signal is directly sampled by sequential logic. This can cause unpredictable or unstable output values, potentially propagating errors through your design. The standard way to suppress metastability in digital systems is the use of a 2-stage synchronizer: two flip-flops in series, clocked by the same clock. This mitigates latching hazards, ensuring a stable output for the receiver clock domain.

Concept:

In a 2-stage synchronizer:

- An asynchronous input is first sampled by FF1, where a metastable event might occur.
- A second flip-flop (FF2), also clocked by the local system clock, samples the output of FF1. By this stage, any metastability has (with very high probability) resolved.
- The output of FF2 is safe, stable, and suitable for use by synchronous logic, but with a one-cycle delay.

Application:

2-stage synchronizers are used whenever an asynchronous or cross-domain signal must be safely absorbed into a synchronous logic domain, such as:

- Interfacing push-buttons, UART, and other I/O to system logic
- Clock domain crossing (CDC) in FPGAs, ASICs, and SoCs
- Any digital system requiring robust, glitch-free operation across clock or chip boundaries

Block Diagram of 2-stage Synchronizer:

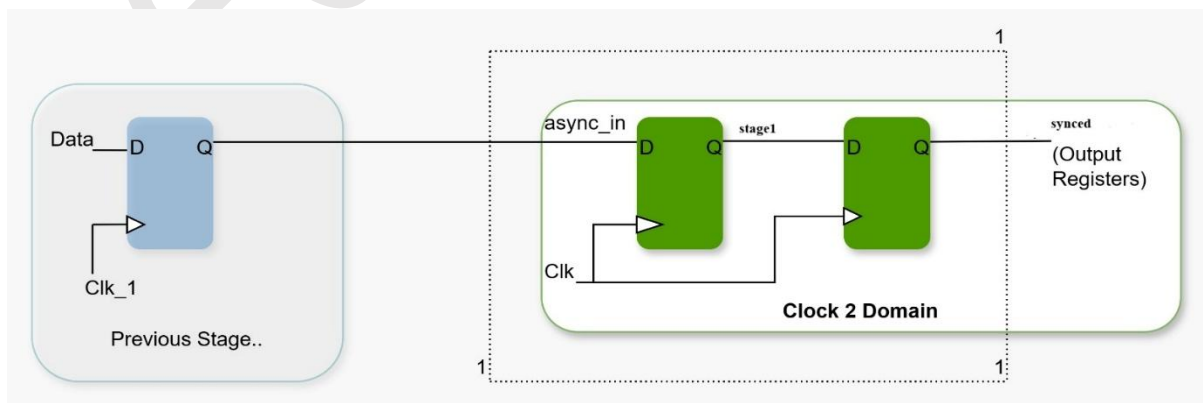


Fig: 2-Stage Synchronizer

Procedure To Create Project:

Step 1: Set Up Project Directory

1.1 Create the lab directory: `mkdir cdc_synchronizer`

1.2: Open directory: `cd cdc_synchronizer`

Step 2: Create RTL Design File

2.1 Create 2-stage synchronizer design file: `cdc_sync.v`

// 2-stage synchronizer module to handle asynchronous input

```
module cdc_synchronizer (  
    input clk,                // Clock input  
    input async_in,           // Asynchronous input signal  
    output reg synced         // Synchronized output signal  
);  
    reg stagel;               // First-stage flip-flop  
    // Two-stage flip-flop chain for synchronization  
    always @(posedge clk) begin  
        stagel <= async_in;  
        synced <= stagel;  
    end  
endmodule
```

2.2 : Save design file.

Step 3: Create Testbench File

3.1 Create testbench file : `gvim cdc_sync_tb.v:`

// Testbench for CDC synchronizer

```
module cdc_sync_tb;  
    reg clk = 0;              // Clock signal  
    reg async_in = 0;         // Asynchronous input  
    wire synced;              // Output from synchronizer  
    // Clock generation  
    always #5 clk = ~clk;
```

```
// Instantiate the DUT
cdc_synchronizer uut (
    .clk(clk),
    .async_in(async_in),
    .syncd(syncd)
);

// Apply transitions not aligned to clk
initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, cdc_sync_tb);
    #12 async_in = 1;
    #7  async_in = 0;
    #6  async_in = 1;
    #9  async_in = 0;
    #50 $finish;
end
endmodule
```

Step 4: Create Simulation Script

4.1 Create script file run.sh:

```
#!/bin/bash

#1: Run Verilator to generate simulation files
verilator --binary -j 0 -Wall cdc_sync.v cdc_sync_tb.v \
    --top cdc_sync_tb --timing --CFLAGS "-std=c++20" --trace

#2: Move into the object directory
cd obj_dir || { echo "obj_dir not found"; exit 1; }

#3: Compile the testbench using make
make -f Vcdc_sync_tb.mk Vcdc_sync_tb || { echo "Compilation
failed"; exit 1; }

#4: Run the testbench simulation
./Vcdc_sync_tb || { echo "Simulation failed"; exit 1; }
```

#5: View the waveform using GTKWave

```
gtkwave dump.vcd
```

4.2 Make the script executable:

```
chmod +x run.sh
```

Step 5: Run and Observe the waveform

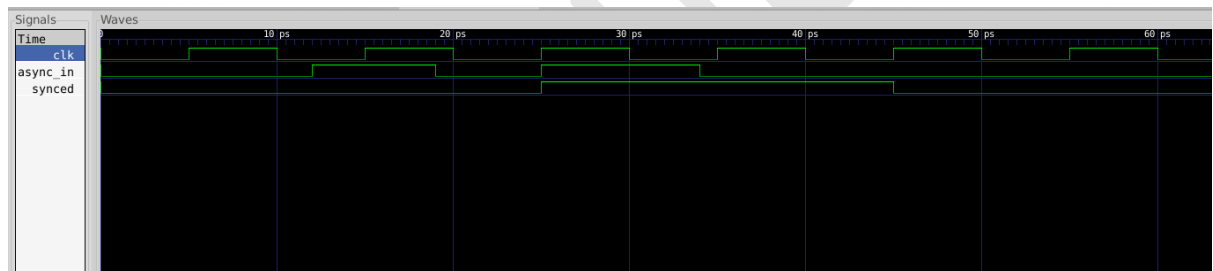
5.1 Run the simulation:

```
./run.sh
```

5.2 Open GTKWave and view signals: clk, async_in, synced

Simulation Output

- **clk**: Regular clock, 10ns period.
- **async_in**: Asynchronously-changed (not clock-aligned).
- **syncd**: Stable output with a one-cycle delay, free from glitches or metastable events.



Observation: You will observe in GTKWave that:

- synced transitions exactly one clock after async_in changes.
- No glitches or temporary pulses are present in synced, regardless of clock when async_in toggles.

Conclusion:

A 2-stage synchronizer is the standard solution for handling metastability and reliably sampling asynchronous inputs. It ensures correct, stable signal transitions into the receiving clock domain, providing robustness and reliability at the cost of a single clock-cycle delay. This lab demonstrates, through simulation, the power and necessity of using proper synchronization when interfacing asynchronous or cross-domain signals in digital system design.